I don't want another Flask-tutorial headline.

The work I actually sign up for when a Python role reaches my desk.

10 slides · swipe

Anton Petnitsky · Python / agents / backend

Real Python work, not tutorial Python.

The roles I reply fastest to:

Backend services behind a real business flow.
LLM pipelines with retries and failure states.
Automation inside messy, unowned workflows.

Frameworks are the easy part.

Anyone can wire a Flask route in an afternoon. The hard part is understanding why the broken path exists — and owning it once it reaches production.

Owning the failure beats knowing the framework.

PYTHON · WHERE IT BREAKS · 4/10

Where Python systems actually fail.

01 Input nobody modeled (bad rows, empty fields)

02 A retry that duplicates a side effect

03 An LLM answering confidently with garbage

04 A background job that died at 3 a.m., silently

05 Config drift between what prod runs and what repo says

None of these show up in a todo app tutorial.

Anton Petnitsky · Python / agents / backend

Python is a tool, not a finish line.

Python is a practical language for solving a broken system, not a credential on a profile.

The value is in what you make survive production — not in knowing syntax.

Anton Petnitsky · Python / agents / backend

The loops I'm best at.

01 Trace the path from user to database



02 Find the step that fails silently



03 Fix it so the failure is visible next time



04 Leave behind a runbook and an alert

One loop like that beats ten greenfield repos.

Anton Petnitsky · Python / agents / backend

What proof looks like for me.

WEAK

Stack jargon

10 frameworks
Buzzwords
screenshots

STRONG

Failure stories

root cause
before/after
with evidence

Hiring? Ask me about the last thing I broke and fixed.

Anton Petnitsky · Python / agents / backend

What I need from a role.

A system someone actually uses. A broken thing I can own.
Roles that end at 'deploy to prod' do not interest me;
roles that start there do.

**Deployment is the beginning
of the interesting part.**

Anton Petnitsky · Python / agents / backend

The kind of problems I say yes to.

01 A pipeline that eats messy exports from a real business

02 An LLM step that must never silently produce nonsense

03 The workflow everyone routes around because it's broken

04 The on-call path that keeps waking someone up

If it's already clean, it doesn't need me. If it's cursed, it does.

Anton Petnitsky · Python / agents / backend

Is that your team?

If your Python work is backend flow, LLM pipelines with failure handling, or the messy automation nobody wants to touch — send me a note about the role.

antonpetnitsky.com — Python / agents / backend